

BÀI THỰC HÀNH: CẤU TRÚC DỮ LIỆU ĐỒ THỊ (GRAPH)

1. MỤC TIÊU BÀI THỰC HÀNH

- Hiểu khái niệm và cấu trúc của đồ thị (graph), phân biệt vô hướng – có hướng, có trọng số – không trọng số.
- Biết các cách cài đặt đồ thị bằng Java: danh sách kề, ma trận kề.
- Cài đặt và thực hiện các thuật toán duyệt đồ thị: DFS và BFS.
- Ứng dụng đồ thị để giải các bài toán thực tế như tìm đường đi, mạng xã hội, sơ đồ luồng công việc.

2. CÁC VÍ DỤ MINH HỌA (CHƯƠNG TRÌNH HOÀN CHỈNH)

Ví dụ 1: Biểu diễn đồ thị bằng danh sách kề

```
import java.util.ArrayList;
import java.util.List;

class Graph {
    private final int V;
    private final List<List<Integer>> adj;

    public Graph(int v) {
        this.V = v;
        adj = new ArrayList<>(V);

        // Khởi tạo danh sách kề cho từng đỉnh
        for (int i = 0; i < V; i++) {
            adj.add(new ArrayList<>());
        }
    }

    public void addEdge(int u, int v) {
        if (u < 0 || u >= V || v < 0 || v >= V) {
```

```
        throw new IllegalArgumentException("Chỉ số đỉnh không hợp lệ!");
    }

    adj.get(u).add(v);
    adj.get(v).add(u); // đồ thị vô hướng
}

public void printGraph() {
    for (int i = 0; i < V; i++) {
        System.out.print("Đỉnh " + i + ": ");
        for (int neighbor : adj.get(i)) {
            System.out.print(neighbor + " ");
        }
        System.out.println();
    }
}

}

public class Main {
    public static void main(String[] args) {
        Graph g = new Graph(5);
        g.addEdge(0, 1);
        g.addEdge(0, 4);
        g.addEdge(1, 2);
        g.addEdge(1, 3);
        g.addEdge(1, 4);
        g.addEdge(2, 3);
        g.addEdge(3, 4);
        g.printGraph();
    }
}
```

```
}  
}
```

Ví dụ 2: Cài đặt đồ thị bằng ma trận kề

```
import java.util.*;  
  
class Graph {  
    private int V;  
    private int[][] adjMatrix;  
    public Graph(int v) {  
        this.V = v;  
        adjMatrix = new int[V][V]; // mặc định = 0  
    }  
    // Thêm cạnh vào ma trận kề  
    public void addEdge(int u, int v) {  
        adjMatrix[u][v] = 1;  
        adjMatrix[v][u] = 1; // đồ thị vô hướng  
    }  
    public void BFS(int start) {  
        boolean[] visited = new boolean[V];  
        Queue<Integer> queue = new LinkedList<>();  
        visited[start] = true;  
        queue.add(start);  
        System.out.print("Duyệt BFS từ đỉnh " + start + ": ");  
        while (!queue.isEmpty()) {  
            int v = queue.poll();  
            System.out.print(v + " ");
```

```

        // Duyệt tất cả đỉnh kề bằng ma trận
        for (int u = 0; u < V; u++) {
            if (adjMatrix[v][u] == 1 && !visited[u]) {
                visited[u] = true;
                queue.add(u);
            }
        }
    }

    System.out.println();
}

public class Main {
    public static void main(String[] args) {
        Graph g = new Graph(5);
        g.addEdge(0, 1);
        g.addEdge(0, 2);
        g.addEdge(1, 3);
        g.addEdge(2, 4);
        g.BFS(0);
    }
}

```

Ví dụ 3: Duyệt đồ thị bằng DFS (Đệ quy)

```

import java.util.ArrayList;

import java.util.List;

class Graph {
    private final int V;

```

```
private final List<List<Integer>> adj;

public Graph(int v) {
    this.V = v;
    adj = new ArrayList<>(V);
    for (int i = 0; i < V; i++) {
        adj.add(new ArrayList<>());
    }
}

public void addEdge(int u, int v) {
    validateVertex(u);
    validateVertex(v);
    adj.get(u).add(v);
    adj.get(v).add(u); // đồ thị vô hướng
}

private void validateVertex(int v) {
    if (v < 0 || v >= V) {
        throw new IllegalArgumentException("Đỉnh " + v + " không hợp lệ!");
    }
}

private void dfsUtil(int v, boolean[] visited) {
    visited[v] = true;
    System.out.print(v + " ");
    for (int u : adj.get(v)) {
        if (!visited[u]) {
            dfsUtil(u, visited);
        }
    }
}
```

```

    }
}

public void DFS(int start) {
    validateVertex(start);

    boolean[] visited = new boolean[V];

    System.out.print("Duyệt DFS từ đỉnh " + start + ": ");

    dfsUtil(start, visited);

    System.out.println();
}
}

public class Main {
    public static void main(String[] args) {
        Graph g = new Graph(5);

        g.addEdge(0, 1);
        g.addEdge(0, 2);
        g.addEdge(1, 3);
        g.addEdge(2, 4);

        g.DFS(0);
    }
}

```

Ví dụ 4: Duyệt đồ thị bằng BFS (Hàng đợi)

```

import java.util.*;

class Graph {
    private int V;

    private List<List<Integer>> adj;

    public Graph(int v) {

```

```

this.V = v;

adj = new ArrayList<>(V);

// Khởi tạo danh sách kề
for (int i = 0; i < V; i++) {
    adj.add(new ArrayList<>());
}
}

public void addEdge(int u, int v) {
    adj.get(u).add(v);
    adj.get(v).add(u); // đồ thị vô hướng
}

public void BFS(int start) {
    boolean[] visited = new boolean[V];

    Queue<Integer> queue = new LinkedList<>();

    visited[start] = true;
    queue.add(start);

    System.out.print("Duyệt BFS từ đỉnh " + start + ": ");

    while (!queue.isEmpty()) {
        int v = queue.poll();

        System.out.print(v + " ");

        for (int u : adj.get(v)) {
            if (!visited[u]) {
                visited[u] = true;
                queue.add(u);
            }
        }
    }
}

```

```

    }
}
System.out.println();
}
}
public class Main {
    public static void main(String[] args) {
        Graph g = new Graph(5);
        g.addEdge(0, 1);
        g.addEdge(0, 2);
        g.addEdge(1, 3);
        g.addEdge(2, 4);
        g.BFS(0);
    }
}

```

3. NỘI DUNG BÀI THỰC HÀNH

Bài 1 (nâng cao): Đếm số thành phần liên thông

- Với mỗi đỉnh chưa duyệt, gọi DFS/BFS.
- Đếm số lần gọi → chính là số thành phần liên thông trong đồ thị.

Bài 2 (ứng dụng): Mô phỏng mạng xã hội

- Mỗi đỉnh là một người dùng.
- Một cạnh biểu thị mối quan hệ bạn bè.
- Cho biết có bao nhiêu nhóm bạn bè (liên thông).
- Tìm tất cả bạn bè trong cùng nhóm với người dùng X.